

ROADASSIST BHARAT

Presentation Brief

What to say, in what order, and how to answer the hard questions.

Every figure in this brief was measured on **16 Sep 2026** by running the thing it describes. Nothing here is estimated, and nothing is claimed that the repository cannot demonstrate on request.

1 Open with this

Thirty seconds. Say what it is, then immediately say which half is real. Volunteering the limit first is what buys credibility for everything after it.

RoadAssist Bharat is a roadside-assistance platform for India, built for the roads where coverage is worst. It works on a smartphone, on a feature phone over SMS, and with no network at all. One thing before the demo: the road-damage detection is a genuinely trained YOLO model. The roadside diagnosis is not — it is a deterministic rule table, and every screen that shows it says so. I will show you both.

Why this works. Examiners probe for over-claiming. Disclose the boundary yourself in the first minute and the obvious attack is gone; everything after inherits the credibility.

2 Demo order

Five moves, about eight minutes. Each shows something the previous one could not.

#	Show	The sentence that lands it
1	RAKSHA authority dashboard localhost:4000/raksha.html · +919999900001	"65 of these 71 detections are real output from our trained model — the database records which, so you can check."
2	Verify a detection click Verify on any row	"DETECTED to VERIFIED, and the action becomes Close. It is a state machine, not a mock-up."
3	Android app — hold SOS emulator, already signed in	"Hold, not tap — a pocket press must never call a responder. The fix arrives with six decimal places of GPS."
4	Feature phone over SMS curl the telecom endpoint	"No app, no data. The same emergency, from a phone that costs a thousand rupees."
5	npm run verify in the terminal	"This is the part I am most proud of. It fails the build if a number in our documents disagrees with the code."

The strongest single moment

After the Android SOS lands, switch to the RAKSHA map. The incident is on the **same NH-48 corridor** the dashboard monitors, because the GPS fix is set to Kherki Daula. Citizen emergency and authority monitoring are one system, and you can see it.

3 Five things that make this advanced

Most projects demo features. These are engineering decisions with reasons behind them. Lead with whichever the examiner seems most technical about.

I. The build refuses to let us lie

measured.json is the single source of every number, and records *how* each was obtained.

check-claims.mjs fails the build when any document disagrees. **check-citations.mjs** checks that all 41 `file.ts:123` references still point at real code.

The same number went stale in twenty documents three separate times. Each time a human caught it, which is not a mechanism. So we made it a build failure.

If pushed: a bug survived inside that gate — citations written without a directory were invisible to it. Three had rotted, one pointing two viva packs at a star-rating schema while claiming it was payment verification. The gate now refuses that shape, and coverage went from 30 checked citations to 41.

II. A model is never allowed to dispatch

ADR-0005. A crash signal raises an incident that waits in `AWAITING_CONFIRMATION` until a recorded human, or a corroborating second signal, moves it on.

A false positive that dispatches is worse than a false negative that asks. The model may raise; only a person may commit.

III. Exactly one channel owns an emergency

The SOS ladder falls back data → SMS → 112 → offline queue. The invariant is that precisely one of them owns the report: a confirmed SMS must not *also* queue an API replay; an unconfirmed one must.

Get that wrong and you send two ambulances to one accident, or none. It is 21 unit tests over pure functions, so it runs without a phone.

IV. Races are settled by the database, not by hope

Two mechanics accepting one job is resolved with `SELECT ... FOR UPDATE` inside a transaction. A duplicate SOS converges on one incident through a unique index on a client-minted reference, not a check-then-write. Settlement is *recorded*, never asserted — a partial unique index makes *paid once* true even under concurrent confirmations.

Anything that is only true because two things did not happen at the same instant is not true. We push those guarantees down into Postgres.

V. Off-Grid Mode is the architecture, not a fallback

ADR-0009. The client distinguishes `ONLINE`, `LIMITED` and `OFF-GRID`. An SOS with no signal becomes a real incident stored on the device with its own reference and GPS fix — and says so, rather than pretending it was sent. On reconnect the journal replays idempotently.

We never tell someone help is coming when it is not. The app says stored on this device, with a reference, or it says nothing at all.

4 Hard questions, and how to answer

The answer to a question about something you did not build is the name of the thing you built instead, then the honest boundary.

They ask	You say
"Is the AI real?"	Half of it. RAKSHA is a trained YOLO11 model — weights and per-epoch metrics exist, best mAP50 0.471. The roadside diagnosis is a rule table returning rules-1.0.0, labelled as such on every screen.
"What is the accuracy?"	mAP50 0.471, mAP50-95 0.226 on held-out validation. Potholes are the hard class at about 0.24 — small objects, under-trained. Loss was still descending when the time cap hit.
"Two mechanics accept at once?"	One wins. SELECT ... FOR UPDATE on the booking row inside the transaction, expiry re-checked under the lock. A concurrency suite races them deliberately.
"How do you stop duplicate payments?"	Settlement is recorded, not asserted. A partial unique index means a second confirmation cannot create a second settled row, so replaying it changes nothing.
"What if the database dies?"	/health returns 503 naming the database; /v1/ping still returns 200 — so the client can tell <i>you have no network from the platform is sick</i> . CI kills Postgres under a running API and asserts exactly that.
"Can the audit log be edited?"	No. It is hash-chained, and Postgres RULES make UPDATE and DELETE no-ops. The chain is re-verified after a backup restore in CI.
"Where is autoscaling?"	Not built, and the slide says DESIGN. What is real is the readiness gate I just showed you.
"Are all eight languages done?"	The strings exist for all eight and a test holds each to its SMS segment budget. They are not native-reviewed — the count is real, the quality is unverified, and those are different claims.

5 Never claim these

Each is false, and each is the kind of thing that unravels a viva once a thread is pulled.

Do not say	The truth
"The AI diagnoses the fault."	It is a rule table. ADR-0006.
"626 tests pass."	636 assertions across six suites. Quote what ran today.
"The payment suite passes."	22 Razorpay checks need a sandbox account and were not run .
"Tested at scale."	No load test. Latency is single-user, on one machine.
"The translations are done."	They exist. They are not native-reviewed.
"iOS / IVR / USSD / satellite."	Designed, not built. Say roadmap.

6 The numbers

Measured 16 Sep 2026 against a database migrated from empty and seeded.

Tests	636 assertions, six suites, 0 failures
-------	---

	108 unit · 189 e2e · 75 concurrency · 74 security · 27 gateway · 163 browser
Other runners	66 Android (0 lint errors) · 39 AI
Not run	22 Razorpay sandbox checks — never counted, never called passing
Schema	56 tables · 138 indexes · 84 unique · 5 migrations
API	64 routes
Model	YOLO11, best mAP50 0.471 , mAP50-95 0.226, pothole class ~0.24
Detections live	71 total, 65 from the real model , 6 simulated
Languages	8 — not native-reviewed

7 If something breaks

Symptom	Do this
Dashboard shows dashes, blank map	You reset without reloading RAKSHA. Run <code>npm run demo:raksha</code> (API must be up).
"Failed to fetch" / "Network unavailable"	You opened the .html file directly. Use <code>http://localhost:4000/...</code>
Dispatch finds no mechanic	Test data is drained. <code>npm run demo:reset</code> , then reload RAKSHA.
App shows sign-in again	Tokens live in memory by design. Sign in again.
Emulator location blank	<code>adb emu geo fix 76.9750 28.4089</code>

Starting from cold

```
cd app
docker compose up -d db
npm run demo:reset
npm start
npm run demo:raksha # second terminal, once the API is up
```

Citizen +919876543210 · Authority +919999900001 · Mechanic +919600000000 · the OTP auto-fills.

Close on this. "Everything I showed you is running on this machine, and every number in our documents is checked by the build. If you want to test a claim, open the file — the line numbers still resolve, because we check those too."